

# Distributed Databases

Replication, Sharding, and Concurrency Control

---

## Workshop 5 – Distributed Systems

Semester II, 2026

Prof. Francisco Hidrobo

---

**Student:** Dario Pomasqui

**Date:** October 1, 2026

**Repository:** [adrianasofiaespinoza/Sistemas-Distribuidos](#)



---

Listing 2: All nodes alive.

## 2 Primary–Replica Replication – Activity 2

---

### 2.1 Configuration Steps

A dedicated replication account was created on the primary:

```
CREATE USER IF NOT EXISTS 'repl'@'%' IDENTIFIED BY 'repl';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';
FLUSH PRIVILEGES;
```

Listing 3: Creating the replication user.

The replica was then pointed at the primary using GTID auto-positioning:

```
STOP REPLICA;
CHANGE REPLICATION SOURCE TO
  SOURCE_HOST='mysql-primary',
  SOURCE_USER='repl',
  SOURCE_PASSWORD='repl',
  SOURCE_AUTO_POSITION=1,
  GET_SOURCE_PUBLIC_KEY=1;
START REPLICA;
```

Listing 4: Replica configuration.

### 2.2 Replication Status Evidence

```
Replica_IO_State: Waiting for source to send event
Source_Host:      mysql-primary
Replica_IO_Running: Yes
Replica_SQL_Running: Yes
Seconds_Behind_Source: 0
Retrieved_Gtid_Set: a97f157f-bdb3-11f1-b391-12c9eefbbc21:1-9
Executed_Gtid_Set:  a97f157f-bdb3-11f1-b391-12c9eefbbc21:1-9
```

Listing 5: SHOW REPLICA STATUS output (relevant fields).

Both `Replica_IO_Running` and `Replica_SQL_Running` report **Yes**, confirming the replication channel is healthy.

### 2.3 Data Replication Test

A row was inserted on the primary and read back from the replica:

```
CREATE TABLE IF NOT EXISTS replication_test
(id INT PRIMARY KEY, note VARCHAR(80));
INSERT INTO replication_test VALUES (1, 'created on primary');
```

Listing 6: Insert on primary.

```
-- On mysql-replica:
SELECT * FROM replication_test;
+----+-----+
| id | note                |
+----+-----+
|  1 | created on primary |
+----+-----+
```

Listing 7: Query on replica – row successfully replicated.

## 2.4 Discussion Question

### Why is the replica read-only?

The replica is configured with `-read-only=ON` to guarantee that all writes enter the system through a single authoritative node. If both nodes accepted independent writes, the binary log streams would diverge: each node would generate its own GTID events, and those events would never be automatically merged. The result would be two separate histories with no built-in reconciliation mechanism, leading to split-brain and permanent data inconsistency.

## 3 Horizontal Sharding – Activity 3

### 3.1 Sharding Rule

The user table is partitioned by a simple modular rule on the primary key:

$$\text{shard}(id) = \begin{cases} \text{Shard 1 (mysql-primary)} & \text{if } id \bmod 2 = 0 \\ \text{Shard 2 (mysql-shard2)} & \text{if } id \bmod 2 = 1 \end{cases}$$

### 3.2 Schema Creation

```
-- Shard 1
CREATE DATABASE IF NOT EXISTS users_shard1;
CREATE TABLE IF NOT EXISTS users_shard1.users (
  id      INT PRIMARY KEY,
  name    VARCHAR(50),
  email   VARCHAR(80),
  city    VARCHAR(50)
);
-- Shard 2 (same DDL, different database)
CREATE DATABASE IF NOT EXISTS users_shard2;
CREATE TABLE IF NOT EXISTS users_shard2.users ( ... );
```

Listing 8: Schema on both shards (identical structure).

### 3.3 Data on Each Fragment

```

SELECT * FROM users_shard1.users ORDER BY id;
+----+-----+-----+-----+
| id | name          | email                      | city      |
+----+-----+-----+-----+
| 2  | Ana Gomez    | ana.gomez@mail.com        | Bogota    |
| 4  | Carlos Ruiz  | carlos.ruiz@mail.com      | Medellin  |
| 6  | Lucia Perez  | lucia.perez@mail.com      | Cali      |
+----+-----+-----+-----+

```

Listing 9: Shard 1 – even IDs (mysql-primary).

```

SELECT * FROM users_shard2.users ORDER BY id;
+----+-----+-----+-----+
| id | name          | email                      | city      |
+----+-----+-----+-----+
| 1  | Mario Lopez  | mario.lopez@mail.com      | Barranquilla |
| 3  | Sofia Torres | sofia.torres@mail.com     | Cartagena    |
| 5  | Diego Castro | diego.castro@mail.com     | Bucaramanga  |
+----+-----+-----+-----+

```

Listing 10: Shard 2 – odd IDs (mysql-shard2).

### 3.4 Reconstructed Logical Result

Since no single SQL engine spans both servers, the coordinator (the application layer) merges both result sets:

| id | name         | email                 | city         | Shard |
|----|--------------|-----------------------|--------------|-------|
| 1  | Mario Lopez  | mario.lopez@mail.com  | Barranquilla | 2     |
| 2  | Ana Gomez    | ana.gomez@mail.com    | Bogota       | 1     |
| 3  | Sofia Torres | sofia.torres@mail.com | Cartagena    | 2     |
| 4  | Carlos Ruiz  | carlos.ruiz@mail.com  | Medellin     | 1     |
| 5  | Diego Castro | diego.castro@mail.com | Bucaramanga  | 2     |
| 6  | Lucia Perez  | lucia.perez@mail.com  | Cali         | 1     |

Table 1: Full user set reconstructed by the coordinator.

### 3.5 Vertical Sharding Challenge

A vertical sharding design splits columns across nodes instead of rows:

- **Node A (identity):** id, name
- **Node B (contact):** id, email, city

To reconstruct a complete user, the application queries both nodes and joins on id. This approach benefits write-heavy workloads where identity fields are updated far less often than contact details, allowing each shard to be placed on hardware tuned for its specific access pattern.

## 4 Lost Update and Locking – Activity 4

### 4.1 4.1 – Reproducing the Lost Update

The accounts table was created with initial balances of 1000 for both Alice and Bob. Two sessions used `READ COMMITTED` isolation and performed a naive read-modify-write cycle:

| Session A                                                                                                                                                                               | Session B                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SET SESSION TRANSACTION<br>ISOLATION LEVEL READ COMMITTED;<br>START TRANSACTION;<br>SELECT balance ... → 1000<br>computes $1000 + 100 = 1100$<br>UPDATE ... SET balance=1100<br>COMMIT; | SET SESSION TRANSACTION<br>ISOLATION LEVEL READ COMMITTED;<br>START TRANSACTION;<br>SELECT balance ... → 1000<br>computes $1000 - 50 = 950$<br><br>UPDATE ... SET balance=950<br>COMMIT; |

Figure 2: Lost update scenario – Session B overwrites Session A.

**Observed final balance: 950.** The mathematically correct value, 1050, was not preserved because Session B read the pre-A value (1000), computed its delta from that stale baseline, and overwrote A's committed change. Under `READ COMMITTED`, each statement sees the latest committed snapshot, but the application-level read and write are not atomic.

### 4.2 4.2 – Fix with `SELECT ... FOR UPDATE`

The balance was reset to 1000. With pessimistic locking, Session A immediately acquires an exclusive row lock:

```
-- Session A
START TRANSACTION;
SELECT balance FROM accounts WHERE id=1 FOR UPDATE; -- acquires
lock
UPDATE accounts SET balance = balance + 100 WHERE id=1;
COMMIT;

-- Session B (blocks at FOR UPDATE until A commits)
START TRANSACTION;
SELECT balance FROM accounts WHERE id=1 FOR UPDATE; -- waits...
UPDATE accounts SET balance = balance - 50 WHERE id=1;
COMMIT;
```

Listing 11: Serialized execution with `FOR UPDATE`.

```
SELECT id, owner, balance FROM accounts WHERE id=1;
+----+-----+-----+
| id | owner | balance |
+----+-----+-----+
| 1  | Alice |    1050 |
```

```
+-----+-----+-----+-----+
```

Listing 12: Final balance after locking fix.

The final balance is **1050**, as expected. Session B was forced to wait for A to release the lock before reading the already-updated value (1100), so it correctly computed  $1100 - 50 = 1050$ .

## 5 Read-After-Write Consistency – Activity 5

### 5.1 Asynchronous Read

A write was performed on the primary and the replica was queried immediately:

```
-- PRIMARY
UPDATE accounts SET balance=balance+1 WHERE id=1;
SELECT id,balance,updated_at FROM accounts WHERE id=1;
+-----+-----+-----+-----+
| id | balance | updated_at          |
+-----+-----+-----+-----+
| 1  |    1051 | 2026-10-01 16:20:07 |

-- REPLICA (immediate read)
SELECT id,balance,updated_at FROM accounts WHERE id=1;
+-----+-----+-----+-----+
| id | balance | updated_at          |
+-----+-----+-----+-----+
| 1  |    1051 | 2026-10-01 16:20:07 |
```

Listing 13: Write on primary, immediate replica read.

On this fast local machine the replica had already caught up; the read was consistent. In production over a wide-area link, a stale value (1050) would be observable during the replication lag window.

### 5.2 GTID-Based Monotonic Read

The primary's committed GTID set was captured:

```
SELECT @@GLOBAL.gtid_executed;
a97f157f-bdb3-11f1-b391-12c9eefbbc21:1-23
```

Listing 14: GTID set on primary.

The replica was then asked to wait until it had applied that exact set:

```
SELECT WAIT_FOR_EXECUTED_GTID_SET(
  'a97f157f-bdb3-11f1-b391-12c9eefbbc21:1-23', 5);
-- Returns: 0 (set fully applied within 5 s)
```

Listing 15: WAIT\_FOR\_EXECUTED\_GTID\_SET on replica.

**Result: 0** – the replica had applied the full GTID set before the timeout. A return value of 0 guarantees that any subsequent read on the replica will reflect at least the writes captured in that GTID set, satisfying a *read-your-writes* requirement: after a client issues a write, it can obtain the primary’s GTID and pass it to the next read request; the read is routed to a replica only after `WAIT_FOR_EXECUTED_GTID_SET` returns 0.

## 6 Deadlock and Prevention – Activity 6

### 6.1 6.1 – Reproducing the Deadlock

Two Python threads connected to the primary and acquired locks in opposite orders:

| Session A                            | Session B                            |
|--------------------------------------|--------------------------------------|
| START TRANSACTION;                   | START TRANSACTION;                   |
| UPDATE ... WHERE id=1 → locks row 1  |                                      |
|                                      | UPDATE ... WHERE id=2 → locks row 2  |
| UPDATE ... WHERE id=2 → <b>waits</b> |                                      |
| COMMITTED                            | UPDATE ... WHERE id=1 → <b>waits</b> |
|                                      | <b>ERROR 1213 (victim)</b>           |

Figure 3: Deadlock scenario – circular lock dependency.

```
[A] Locked row 1, waiting 2s before locking row 2...
[B] Locked row 2, waiting 2s before locking row 1...
[B] ERROR 1213: Deadlock found when trying to get lock;
    try restarting transaction
[A] COMMITTED

Session A result: COMMITTED
Session B result: ERROR 1213: Deadlock found when trying
    to get lock; try restarting transaction
```

Listing 16: Observed deadlock output.

MySQL’s deadlock detector selected Session B as the victim (it had performed fewer row-level operations) and rolled it back automatically, releasing all of B’s locks so Session A could proceed.

### 6.2 6.2 – Prevention with Consistent Locking Order

Both sessions were rewritten to acquire an exclusive lock on *all relevant rows in ascending id order* before performing any write:

```
START TRANSACTION;
SELECT * FROM accounts
  WHERE id IN (1,2) ORDER BY id FOR UPDATE;
-- then perform the session's specific updates
COMMIT;
```

Listing 17: Ordered lock acquisition in both sessions.

```
[A] Acquired locks on rows 1,2.
[B] Attempting to lock rows 1,2 in order (will block)...
[A] COMMITTED
[B] Acquired locks. Rows: [(1, 'Alice', 1010, ...), (2, 'Bob', 990, ...)]
[B] COMMITTED

Session A result: COMMITTED
Session B result: COMMITTED
```

Listing 18: No deadlock – Session B blocks, then commits.

### 6.3 Blocking vs. Deadlock

**Blocking** occurs when one transaction holds a lock that another transaction needs. The second transaction waits until the first commits or rolls back. It resolves *automatically* and is an expected part of concurrent operation.

**Deadlock** occurs when two (or more) transactions each hold a lock the other needs, forming a *circular dependency* that can never resolve on its own. The database must intervene by aborting one transaction.

By always acquiring multiple row locks in the same canonical order, the circular dependency is structurally impossible: if A holds lock 1 and wants lock 2, B – which also wants lock 1 first – will simply block waiting for lock 1, breaking the cycle.

## 7 Discussion

Replication, sharding, and concurrency control each address a distinct dimension of the distributed-database problem:

**Replication** increases *availability* and *read throughput*. By maintaining a continuously updated copy of the primary's data, the system can serve read traffic from the replica and survive a primary failure (after a failover procedure). The trade-off is eventual consistency: asynchronous replication introduces a lag window during which replicas may serve stale reads. GTID-based mechanisms such as `WAIT_FOR_EXECUTED_GTID_SET` allow the application to opt in to stronger guarantees selectively.

**Fragmentation (sharding)** improves *write scalability* and *storage capacity*. Horizontal sharding distributes rows across independent nodes, so each shard handles a fraction of the write load and stores a fraction of the data. Vertical sharding splits columns, enabling separate scaling for different access patterns. Both strategies require the application layer (or a middleware coordinator) to route queries and merge partial results, adding operational complexity.

**Concurrency control** preserves *data integrity* under parallel access. Without it, interleaved read-modify-write cycles produce lost updates (as shown in Activity 4), and unordered multi-row locking produces deadlocks (Activity 6). Pessimistic locking (`SELECT ...FOR UPDATE`) serializes conflicting transactions at the cost of reduced concurrency;

optimistic schemes (MVCC, OCC) allow more parallelism at the cost of occasional abort-and-retry.

Together, the three mechanisms operate at complementary layers: replication handles *node-level* redundancy and read scaling; sharding handles *data-level* partitioning; and concurrency control handles *transaction-level* correctness. A production distributed database typically employs all three simultaneously.